

新增模型添加CE流程

1.模型提供

模型上传 aistudio **repo_id** 如： PaddleFormers/tiny-random-gptoss。

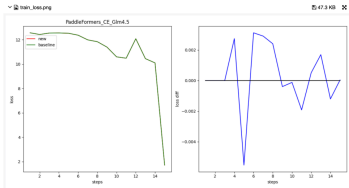
- 模型上传联系 @张依潮
- 模型大小原则不超过300MB

2.自测list

CE 环境： A 100 80G python 3.10 cuda 12.6

Paddle whl : https://paddle-qa.bj.bcebos.com/paddle-pipeline/Develop-TagBuild-Training-Linux-Gpu-Cuda12.6-Cudnn9.5-Trt10.5-Mkl-Avx-Gcc11-SelfBuiltPypiUse/latest/paddlepaddle_gpu-0.0.0-cp310-cp310-linux_x86_64.whl

数据集： 使用CONFIG_YAML中默认数据

1	分类	CONFIG_YAML	训练	热启	合并权重	vllm推理	loss 精度校验	研发自测结论
2	pt	Paddleformers/examples/config/pt/full.yaml			/		训练、热启跑两次 loss能够逐位对齐  提供 loss 对比图或者者 log.txt	测试通过 

3	pt-tp-pp	Paddleformers/examples/config/pt/full_tp_pp.yaml					
4	pt-ep	Paddleformers/examples/config/pt/full_tp_pp_ep.yaml					
5	pt-lora	Paddleformers/examples/config/lora.yaml					测试不通过 ❌ 给出原因
6	pt-lora-tp-pp	Paddleformers/examples/config/pt/lora_tp_pp.yaml					
7	pt-lora-ep	Paddleformers/examples/config/pt/lora_tp_pp_ep.yaml					
8							
9	sft	Paddleformers/examples/config/sft/full.yaml					
10	sft-tp-pp	Paddleformers/examples/config/sft/full_tp_pp.yaml					
11	sft-ep	Paddleformers/examples/config/sft/full_tp_pp_ep.yaml					
12	sft-lora	Paddleformers/examples/config/sft/lora.yaml					
13	sft-lora-tp-pp	Paddleformers/examples/config/sft/lora_tp_pp.yaml					

14	sft-lora-ep	Paddleformers/examples/config/sft/lora_tp_pp_ep.yaml						
15	sft-function-call	Paddleformers/examples/config/sft/full_function_call.yaml						
16								
17	dpo	Paddleformers/examples/config/dpo/full.yaml						
18	dpo-tp-pp	Paddleformers/examples/config/dpo/full_tp_pp.yaml						
19	dpo-ep	Paddleformers/examples/config/dpo/full_tp_pp_ep.yaml						
20	dpo-lora	Paddleformers/examples/config/dpo/lora.yaml						
21	dpo-lora-tp-pp	Paddleformers/examples/config/dpo/lora_tp_pp.yaml						
22	dpo-lora-ep	Paddleformers/examples/config/dpo/lora_tp_pp_ep.yaml						
23	dpo-function-call	Paddleformers/examples/config/dpo/full_function_call.yaml						

原则上新增模型要满足上述所有训练配置！

- 训练参考：

```
</> Bash |  
  
1 环境变量：  
2 export NVIDIA_TF32_OVERRIDE=0  
3 export NCCL_ALGO=Tree  
4 export FLAGS_embedding_deterministic=1  
5 export FLAGS_cudnn_deterministic=1  
6  
7 执行方式：  
8 paddleformers-cli train "$CONFIG_YAML" \  
9 model_name_or_path=$repo_id \  
10 output_dir output/$repo_id \  
11 save_checkpoint_format=flex_checkpoint \  
12 load_checkpoint_format=flex_checkpoint \  
13 unified_checkpoint=false \  
14 max_steps 10 \  
15 save_steps 10 \  
16 fuse_attention_qkv=true \  
17 fuse_attention_ffn=true \  
18 (注意，需要check组网支持fuse qkv/ffn)
```

- 热启参考

```
</> Bash |  
  
1 环境变量：  
2 export NVIDIA_TF32_OVERRIDE=0  
3 export NCCL_ALGO=Tree
```

```
4 export FLAGS_embedding_deterministic=1
5 export FLAGS_cudnn_deterministic=1
6
7 执行方式:
8 paddleformers-cli train "$CONFIG_YAML" \
9 model_name_or_path=$repo_id \
10 output_dir output/$repo_id \
11 save_checkpoint_format=flex_checkpoint \
12 load_checkpoint_format=flex_checkpoint \
13 unified_checkpoint=false \
14 max_steps 12 \
15 save_steps 1000 \
16 fuse_attention_qkv=true \
17 fuse_attention_ffn=true \
```

- 合并权重 (lora)

</>

Bash

```
1 paddleformers-cli export "./examples/config/run_export.yaml" \
2 lora=True \
3 model_name_or_path=$repo_id \
4 output_dir=output/$repo_id
```

- vllm推理 (使用paddleformers训练后的权重)

</>

Bash

```
1 export VLLM_WORKER_MULTIPROC_METHOD=spawn
2 export VLLM_DISTRIBUTED_PORT=10034
3
4 unset PADDLE_ELASTIC_JOB_ID
```

```
5 unset PADDLE_TRAINER_ENDPOINTS
6 unset DISTRIBUTED_TRAINER_ENDPOINTS
7 unset FLAGS_START_PORT
8 unset PADDLE_ELASTIC_TIMEOUT
9 unset PADDLE_TRAINERS_NUM
10 unset PADDLE_TRAINER_ID
11 unset POD_IP
12 python -m vllm.entrypoints.openai.api_server \
13     --model output/$repo_id \ # 换成训练后保存好的ckpt
14     --served-model-name ? \ # 不需要改
15     --tensor_parallel_size 8 \ # 建议改成gpu数量
16     --max-model-len ? \ # 不需要改
17     --port 8777 \ # 部署服务后的端口，不需要改
```

推理代码参考

</>

Bash

```
1 from openai import OpenAI
2 import json
3 import time
4 from tqdm import tqdm
5
6 # 连接到你的 vllm OpenAI 兼容服务
7 client = OpenAI(
8     base_url="http://localhost:8777/v1", # 端口与部署时的 --port 一致
9     api_key="dummy" # vllm 不验证 API key, 任意字符串即可
10 )
11
12 # 1. 文本生成 (适合非对话模型, 对应 completions 接口)
```

```
13 def test_completion():
14     response = client.completions.create(
15         model="glm4moe", # 与部署时的 --served-model-name 一致
16         prompt="请介绍一下大语言模型的应用场景",
17         max_tokens=300,
18         temperature=0.7,
19         top_p=0.95
20     )
21     print("Completion 结果: ")
22     print(response.choices[0].text.strip())
23
24 # 2. 对话交互 (适合对话模型, 对应 chat/completions 接口)
25 def test_chat_completion():
26     #可替换成自己的数据集
27     input_file = '/root/paddlejob/workspace/env_run/evaluation/gsm8k/gsm8k.json'
28     output_file = '/root/paddlejob/workspace/env_run/etion/gsm8k/gsm8k_result_base_sp.json'
29     with open(input_file, 'r', encoding='utf-8') as f:
30         total_lines = sum(1 for _ in f)
31     with open(input_file, 'r', encoding='utf-8') as in_f, \
32         open(output_file, 'w', encoding='utf-8') as out_f:
33
34         for line_num, line in tqdm(enumerate(in_f, 1), total=total_lines, desc="处理进度"):
35             try:
36                 start_time = time.time()
37                 # 解析JSONL中的一行数据
38                 data = json.loads(line.strip())
39
40                 # if line_num < 1211 or line_num > 1234:
41                 #     out_f.write(json.dumps(data, ensure_ascii=False) + '\n')
```

```
42         #         continue
43
44         # 从数据中获取提示词（假设输入JSON包含"prompt"字段）
45         if data[0]['role'] == 'user':
46             prompt = data[0]['content']
47         else:
48             print(f"第{line_num}行role不是user，跳过处理")
49             continue
50
51         # 发送请求
52         response = client.chat.completions.create(
53             model="glm4moe",
54             messages=[{"role": "user", "content": prompt}],
55             max_tokens=3596,
56             temperature=0.0
57         )
58
59         # 提取结果
60         result = response.choices[0].message.content.strip()
61         # print(f"第{line_num}行处理完成，耗时{time.time()-start_time}s")
62
63         # 构建输出数据（包含原始数据和结果）
64         data.append({"role": "pred_answer", "content": result})
65
66         # 写入输出文件（JSONL格式，每行一个JSON对象）
67         out_f.write(json.dumps(data, ensure_ascii=False) + '\n')
68
69     except Exception as e:
70         print(f"第{line_num}行处理出错：{str(e)}")
```



```
71         # 可以选择记录错误信息到输出文件
72         error_data = {** data, "error": str(e)} if 'data' in locals() else {"line_num": line_num,
73         "error": str(e)}
74         out_f.write(json.dumps(error_data, ensure_ascii=False) + '\n')
75
76 # 测试调用
77 if __name__ == "__main__":
78     # test_completion()
79     test_chat_completion()
80     print("done")
```

3.特殊配置组合提供方式

如果有以上自测list没有覆盖到且需要测试的新 config yaml , 可以提交到 examples/config @张依潮 review , 执行方式同上

4.提供提测文档

全部自测试完成之后提供提测文档 @张依潮 review